

Unit-6: Software Testing and Quality Assurance

Verification Vs Validation:

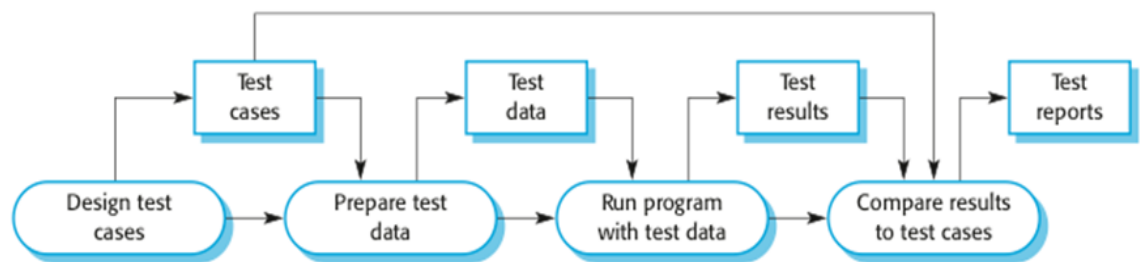
Verification	Validation
Are we building the product right?	Are we building the right product?
Ensures software is built correctly per requirements and standards.	Ensures software meets end-user needs & expectations
Comes before validation	Comes after verification
Activity involves are inspections, review and walkthroughs	Activity involves are testing, prototyping, and user acceptance
Typically performed by developers, QA team	Typically performed by testers, end-users
Example (Layman): Checking the blueprint of a house to see if it matches the architect's plan before construction.	Example (Layman): Inspecting the finished house to see if it's comfortable, functional, and fits the family's needs.

Testing:

- Process of executing a program to identify errors and to ensure that it satisfies the specified requirements.
- **Importances of Software Testing:**
 - Improves software quality, reliability, and performance
 - Prevents failures and lowers maintenance cost
 - Increases user confidence
- **Principles of Software Testing:**
 - **Testing should be based on user requirements:** Focus on validating that the software meets customer needs and works as intended.
 - **Testing time and resources are limited:** Avoid redundant tests; prioritize critical areas.
 - **Exhaustive testing is impossible:** Cannot test every input or path due to large data and complex program flows.
 - **Use effective resources to test:** Employ the right tools, methods, and skilled personnel for testing.
 - **Test planning should be done early:** Begin planning as soon as requirements are defined, even before coding starts.

➤ Software Testing Process:

- **Design Test Cases:** First, we create a set of test cases (defines what to test, the inputs to use, the actions to perform, and the expected outcome).
- **Prepare Test Data:** For each test case, we prepare the required input data needed to execute the test.
- **Run Program with Test Data:** The software is executed using the prepared test data as inputs.
- **Compare Results to Test Cases:** The actual output generated by the software is compared against the expected output defined in the test cases.
- **Generate Test Reports:** The outcomes (pass/fail) of all the comparisons are documented in a test report.



➤ Level of Testing:

- **Unit Testing:**
 - Each individual units or components of a software are tested
 - Purpose is to validate that each unit of the software code performs as expected
 - **Example:**

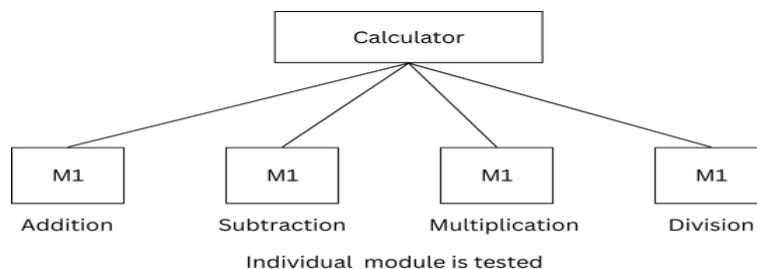
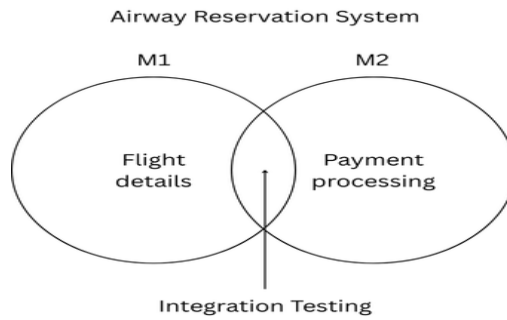


Fig: Unit Testing

- **Integration Testing:**

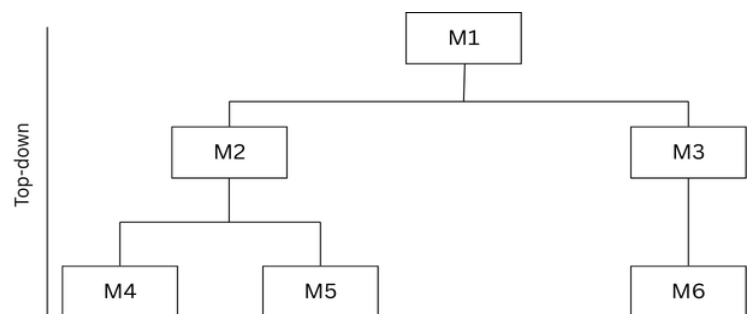
- Two or more modules of an application are logically grouped together and tested as a whole
- Purpose is to find the defects on interface, communication, and data flow among integrated units
- **Example:**



- **Types:**

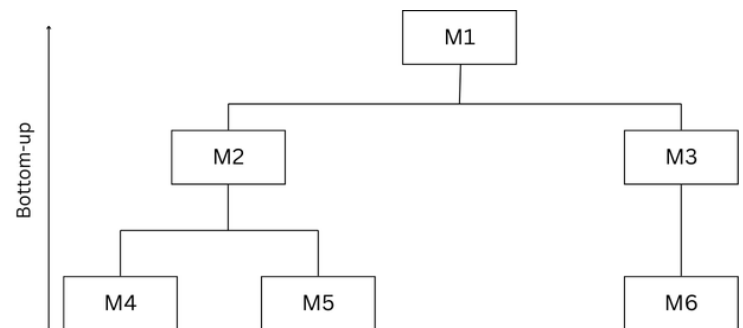
- **Top-Down Integration:**

- Testing takes place in top-down fashion
 - Testing starts with the highest-level modules and then proceeds step by step to lower-level modules.
 - **Stubs** are used to simulate lower-level modules that are not yet developed.
 - **Advantages:**
 - Early testing of high-level logic and major control flow.
 - Useful when high-level functionality is critical.
 - **Disadvantages:**
 - Basic functionality is tested at the end of cycle
 - Can be slow if many stubs are required.



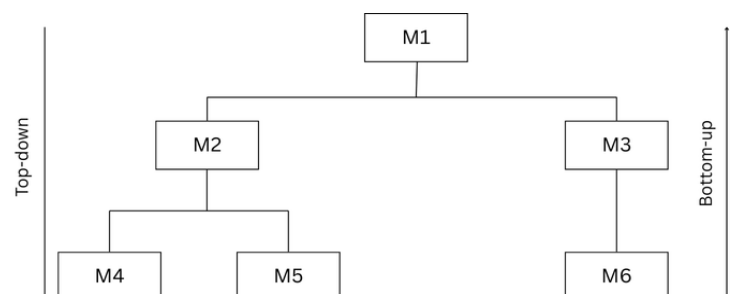
- **Bottom-Up Integration:**

- Testing takes place in bottom-up fashion
- Testing starts with the lowest-level modules (unit testing) and then proceeds step by step to higher-level modules.
- **Drivers** are used to simulate higher-level modules that are not yet developed.
- **Advantages:**
 - Early detection of defects in low-level modules.
 - Ensures lower-level modules are robust before integration.
- **Disadvantages:**
 - Defects in high-level design may be detected late.



- **Sandwich/Hybrid Integration:**

- Combines both top-down and bottom-up approaches for testing.
- Testing starts simultaneously from high-level modules (using stubs) and low-level modules (using drivers) and meets in the middle.
- **Advantages:**
 - High-level and low-level modules tested simultaneously
 - Combines benefits of top-down and bottom-up approaches.
- **Disadvantages:**
 - More complex to plan and execute.
 - Requires both stubs and drivers.



- **Interface Testing:**

- Testing done to check whether **different modules or systems interact correctly**.
- Ensures **data is passed accurately** and communication between components is reliable.
- Main aim is to verify that **interfaces between modules/systems function correctly** and **data is exchanged properly** without errors.
- **Activities Performed:**
 - Checking data transfer between modules.
 - Ensuring protocols, APIs, and message formats are correct.
 - Testing compatibility between interacting systems.

Note: Integration testing checks if modules work together correctly, while interface testing checks if the communication and data exchange between modules are correct.

- **System Testing:**

- Once all the components are integrated, the application as a whole is tested to see that it meets the quality standards
- Main aim is to verify that the system to be delivered meet the specification & purpose
- **Types:**

Black Box Testing	White Box Testing
Main aim is to check it meets the user requirements as per the SRS document	Main aim is to check all program statement as per the design specification
Programming knowledge isn't required	Programming knowledge is required
Generally done by the testers	Generally done by the developers
No need to know about the internal design of the code	Should know about the internal design of the code
Code remains hidden	Code remains visible

- **Regression Testing:**

- Testing done to ensure that new code changes do not break existing functionality.
- Re-runs old test cases every time a change is made to the program.
- Tests must run 'successfully' before the change is committed.

- **User Acceptance Testing:**

- After system testing fixes most defects, the software is given to users for acceptance testing
- Main aim is to check whether the application meets the intended specifications and satisfies the customer's requirements.
- Ensures the product is ready for real-world use.
- **Can be performed in 2 phases:**

Alpha Testing	Beta Testing
Main aim is to Identify bugs and issues early before release	Test how the software performs in real-world scenarios
Performed before beta testing	Performed after alpha testing
Done by internal team (developers & testers)	Done by real users in real environment
Performed at the developer's site	Performed at the end-user's place
Checks spelling errors, broken links, and performance on low-spec machines	Users run the app and give feedback on errors, flow, and crashes

Test Plans:

- Document defining objectives, scope, methods, and purpose of testing.
- Guides how testing will be done, identifies features to test, tasks, required staff, and test environment.
- **IEEE Standard:** Ensures plan is complete, clear, consistent, and verifiable; incomplete plans may cause failure.
- **Main Sections of IEEE (Institute of Electrical and Electronics Engineers):**
 - **Overview:** Summary of software and testing purpose
 - **Test Scope:** Features to test and exclude
 - **Test Methodologies:** Testing approaches and techniques
 - **Test Phases:** Stages of testing
 - **Test Environment:** Setup in which testing is performed
 - **Schedule:** Testing timeline and milestones
 - **Approvals and Distribution:** Stakeholders who approve and receive the plan

Test Case:

- Set of actions to verify a specific feature or functionality of software
- Document containing set of test data, pre-conditions, expected results, and post-conditions
- Developed for a particular **test scenario** in order to make sure the software works as per requirements
- **Test Scenario:** High-level description of functionality to be tested.

E.g.: Verify that a registered user can successfully login to the e-commerce app.

- **Typical Test Case Parameters:**

Test Case ID, Test Scenario, Test Case Description, Test Steps, Pre-requisite, Test Data, Expected Output, Actual output, Environment Information, Comments

- **Test Data:** Input (both positive and negative) given to a software program during test execution.

Test Scenario ID		TS_EC_001	Test Case ID		TS_EC_001		
Test Case Description		Verify that a registered user can successfully login to the e-commerce application.	Test Priority		High		
Pre-Requisite		A valid user account	Post-Requisite		N/A		
Test Execution Steps:							
S.N.	Action	Inputs	Expected Output	Actual Output	Test Browser	Test Result	Test Comments
1.	Open the e-commerce website login page	URL: www.mye-commerce.com/ login	Login page should be displayed	Login page displayed	Chrome	Pass	-
2	Enter valid email and password	Email: testuser@gmail.com Password: Test@123	Credentials accepted	Credentials accepted	Chrome	Pass	-
3	Click on “Login” button	-	User should be redirected to homepage/dash board	Redirected to homepage	Chrome	Pass	-

Table 1: Sample for Test Case Template

Design of Test Cases:

- Creating a set of actions, inputs, and expected outputs to verify that a system or component works correctly
- **Why Design Test Cases?**
 - To guide testers on what to test and how.
 - To detect defects early and ensure quality.
 - To help estimate testing effort and resources.
 - To serve as documentation for future reference and audits.
- **Various approaches that you can take to designing the test case:**
 - **Requirements-based Testing:**
 - Designing test cases based solely on documented specifications and requirements without considering the internal structure of the code
 - Used to verify that the software's behavior matches the specified requirements.
 - **Example:**
 - **Requirement:** "The system shall allow login only when the user ID is 6-12 alphanumeric characters."
 - **Test Cases:**
 - **Valid:** Test with "User123" (9 chars). Expected: Login success.
 - **Invalid:** Test with "Usr1" (4 chars). Expected: Error message.
 - **Boundary:** Test with "User12" (6 chars) and "User12345678" (12 chars). Expected: Login success.
 - **Partition Testing:**
 - Inputs/Outputs are grouped into partitions, and tests are designed to cover each partition.
 - If one value in a partition works (or fails), all other values in that same partition should behave identically (no need to test every single value).
 - Optimize test coverage while minimizing redundant tests.
 - **Main Techniques:**
 - **Equivalence Partitioning (EP):** Tests one general value from the middle(non-boundary) of each group.
 - **Boundary Value Analysis (BVA):** It focuses on testing the values at the edges of partitions.

- **Example:**
 - **Input Field:** "Age" (must be between 18 and 65 to register).
 - **Partitions:** [<18], [18-65], [>65]
 - **Test Values (using BVA):** 17, 18, 19, 64, 65, 66
 - **Test Values (using EP):** 10,40,70
- **Structural Testing:**
 - Writing tests based on how the code is built, not just what it does.
 - Verifies code quality, robustness, and efficiency
 - What to measure?
 - **Statement Coverage:** Has every line of code been run?
 - **Branch Coverage:** Has every possible branch (e.g., every outcome of an if-else statement) been taken?
 - **Path Coverage:** Has every possible path through a given part of the code been executed?
- **Example:**
 - **Code Snippet:**

```

if user_role == "admin":
    grant_full_access()
else:
    grant_limited_access()

```
 - **Structural Test Cases:**
 - Set user_role = "admin" -> Verify grant_full_access() is called.
(Tests the True branch)
 - Set user_role = "user" -> Verify grant_limited_access() is called.
(Tests the False branch)

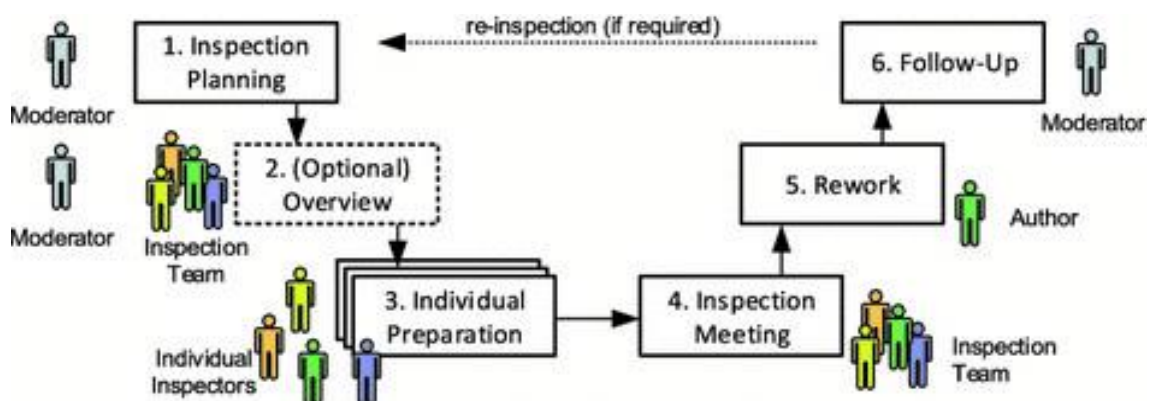
This achieves 100% Branch Coverage.

Software Inspections:

- A formal peer-review process, introduced by IBM in the early 1970s, where team members examine a developer's work (code, documentation, design, etc.) to detect defects early.
- **Benefits of Software Inspection:**
 - Detects defects early in development, reducing costly rework and delays
 - Enhances code readability, maintainability, efficiency, and security
 - Promotes collaboration and knowledge sharing within the team

Software Inspections Process:

- **Planning:** Moderator plan the inspection.
- **Overview Meeting:** Author explains the work product.
- **Preparation:** Inspectors carefully study the product alone and list defects before the group meeting.
- **Inspection Meeting:** Team discusses and records defects.
- **Rework:** Author fixes the identified issues.
- **Follow Up:** Moderator verifies corrections are done.



Quality Management Activities:

- Set of coordinated tasks and practices aimed at ensuring that a software product meets defined quality standards.
- **SQM can be structured into 3 main activities:**
 - **Software Quality Planning:**
 - The process or plan with the help of which certain quality objectives can be achieved
 - Often, done by the organization before starting the project
 - **Activities performed during quality planning:**
 - **Defining Quality Goals:** Setting clear goals for the quality of what the project will deliver
 - **Identify Quality Standards:** Selecting relevant industry standards and best practices to meet expectation
 - **Developing Quality Metrics:** Defining metrics and indicators to measure quality
 - **Allocating Resources:** Identify tools and people needed to ensure quality
 - **Risk Management:** Identify and mitigate quality risks
 - **Continuous Improvement:** Continuously enhance quality planning by learning from each project

- **Software Quality Assurance Vs Software Quality Control:**

Software Quality Assurance	Software Quality Control
Set of activities for ensuring quality in the processes by which products are developed	Set of activities for ensuring quality in products, focused on identifying defects in the products produced
Process Oriented	Product Oriented
All the team member are responsible	Dedicated inspection or testing team is responsible
Done before Quality Control	Done only after Quality Assurance
Some activities include process audits, process monitoring, trainings and feedback collection	Some activities include testing, inspections, reviews, and defect management

Product and Process Quality:

- Software quality depends on the development process.
- Product quality is measured to continuously improve the development process.
- Experienced engineers enhance both process and product quality.
- Measuring software quality attributes (i.e. maintainability, reliability, and usability) and understanding how the process characteristics affects them is hard.
- However, experience has shown that, a high-quality process leads to better software products.

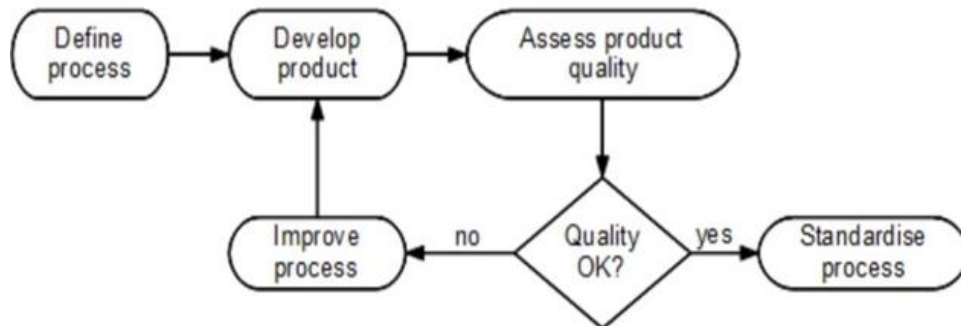


Fig: Process based quality assessment

Quality Assurance Standards

- **Product Standards:** Applied to the final software product (output).
- **Process Standards:** Define the procedures to follow during development.
- **2 main types of QA standards:**
 - ISO (International Organization for Standardization)
 - Documentation standards
- **ISO 9000 family:**
 - The ISO 9000 family is a **set of international standards that guide organizations in building Quality Management Systems (QMS) to consistently meet customer and regulatory requirements.**
 - **ISO 9000:2015 Seven Quality Management Principles:**
 - **Customer Focus:** Understand and meet customer needs, aiming to exceed expectations.
 - **Leadership:** Establish a clear vision, direction, and create an engaging environment.
 - **Engagement of People:** Involve all employees to fully utilize their skills and knowledge.
 - **Process Approach:** Manage activities as interrelated processes for efficient results.
 - **Improvement:** Make continual improvement a permanent organizational goal.
 - **Evidence-Based Decision Making:** Use data and analysis to guide decisions.
 - **Relationship Management:** Build mutually beneficial relationships with stakeholders and suppliers.
 - 5 standards in the ISO 9000 series are:
 - **ISO 9000-1:** Provides the fundamental concepts and guidelines for selecting and using the other standards in the series.
 - **ISO 9001:** The comprehensive quality assurance standard for design, development, production, installation, and servicing.
 - **ISO 9002:** A now-obsolete model for quality assurance in production, installation, and servicing (excluding design).
 - **ISO 9003:** A now-obsolete model for quality assurance in final inspection and testing only.
 - **ISO 9004-1:** Provides guidelines for implementing a comprehensive quality management system to achieve business performance improvement.

➤ **Documentation standards:**

- Guidelines that define how documents related to software development and quality assurance should be prepared, organized, and maintained.

Capability Maturity Model (CMM):

- Framework used to measure and improve the quality and maturity of an organization's software development processes
- Assesses an organization's current process maturity level and provides a clear roadmap to reach the next level of improvement.

➤ **5 Maturity Levels of CMM:**

○ **Initial Level (Process unpredictable and poorly controlled):**

- No engineering management, everything done without a proper system.
- Software process is unpredictable with respect to time and cost
- It depends on current staff, as staff change so does the process

○ **Repeatable Level (Basic project management):**

- Planning and management of new projects is based on the experience with the similar projects
- Realistic plans are made using performance from past projects.

○ **Defined Level (Process standardization):**

- Process of developing and maintaining s/w across the organization is documented including engineering and management
- Training programs are implemented to ensure that the staff have skills and knowledge required
- Basic Risk Management

○ **Managed Level (Quantitative measurement):**

- Organization set quantitative goals for both product and process
- Here process is predictable both with respect to time and cost

○ **Optimizing Level (Continuous process improvement):**

- Organization analyzes defects to determine their cause and set goals to prevent their recurrence
- Company continuously improves the process performance of their project

